

FRC RAG Application Documentation

Section 1 – Quick links:

- [GitHub Repository](#)
- [RAGAS Results](#)

Section 2 – Documentation:

- JS Documentation:
 - *POST - /api/query*
 - Sends request with user information to the Python route: **/api/query**
 - *GET - /*
 - Renders HTML Frontend
- Python Documentation (Models):
 - **Embedding Model:** MiniLM-L6-v2
 - **User-Uploaded Image Processing Model:** Bakllava
 - **Decomposition and Rephrasing Model:** Llama 3.2:3b
 - **Sentiment Classifier:** Llama 3.2:3b
 - **Intent Classifier:** Llama 3.2:3b
 - **User Response Generator:** Qwen 2.5:7b
- Python Documentation (Functions):
 - `clean_ocr_to_markdown_table(raw_ocr_text):`
 - Takes Raw CSV text from the vision model and formats it into a well-formed markdown table that marked.js can render.
 - `perform_vision_ocr(base64_image_string):`
 - Leverages a local vision model to accurately parse text, tables, or diagram rules out of user uploaded images.
 - `decompose_and_rephrase_query(user_query):`
 - Decomposes compound questions into subqueries and rephrases them to maximize vector database embedding matches

- `extract_pdf_with_tables(pdf_path):`
 - Extracts raw text and table structures from ALL pages of the PDF Loops page-by-page with try-except blocks to prevent errors
- `chunk_text(text, chunk_prefix= ""):`
 - Splits chunk into coherent blocks, preserving table structure. Uses delimiters such as newlines, double newlines, periods, and spaces.
- `load_chunks_to_model(chunks, prefix_id, source_type, batch_size=100):`
 - Vectorizes chunks using unique prefixed tracking strings.
 - The prefixed tracking strings indicate from which source the text belongs.
- `generate_database():`
 - Build the database using the manual and team updates.
- `classify_query_sentiment(user_query):`
 - Uses a lightweight model to quickly categorize the user's emotional tone before prompt generation
- **Python Documentation (Routes):**
 - *GET - /*
 - Returns App Status
 - *POST - /api/query*
 1. Stores User Query, Chat History, and Image Bytes
 2. Performs OCR on user-uploaded images using `perform_vision_ocr(base64_image_string)`
 3. Detects User Sentiment using `classify_query_sentiment(user_query)`
 4. Detects User Intent to prevent prompt injection
 - a. If Prompt injection is detected, it displays a fallback answer and ends the execution sequence
 5. Decomposes and rephrases the query using `decompose_and_rephrase_query(user_query)`
 6. Embeds all search queries and returns the top 10 results from the Game Manual and the top 8 results from the Team Updates
 7. Adds Newest Results to retrieved contexts
 8. Re-ranks context based on chunks with a cosine similarity of 0.1 relative to the user prompt
 9. Generates the text output using an LLM and displays the LLM Response